

Введение в C#



- Платформа **.NET** — современная среда разработки от Microsoft
- Используется для создания различных типов приложений
- Основной язык разработки — **C#**



Что такое .NET?

.NET — это программная платформа, разработанная компанией **Microsoft**, которая предназначена для создания различных типов приложений.

Платформа предоставляет разработчикам:

- ❑ среду выполнения программ
- ❑ набор библиотек
- ❑ инструменты разработки
- ❑ поддержку разных операционных систем

С помощью .NET можно создавать:

- ❑ веб-приложения
- ❑ серверные сервисы и API
- ❑ настольные программы
- ❑ мобильные приложения
- ❑ облачные решения.

.NET = платформа разработки



Язык программирования C#

Основным языком разработки на платформе .NET является **C#**.

C# — это современный объектно-ориентированный язык программирования, который сочетает:

- ❑ простоту синтаксиса
- ❑ высокую производительность
- ❑ безопасность типов
- ❑ поддержку современных подходов разработки

Язык C# широко используется для разработки:

- ❑ веб-сервисов
- ❑ корпоративных систем
- ❑ облачных приложений
- ❑ мобильных приложений
- ❑ игр.



Архитектура платформы .NET

Платформа .NET включает несколько ключевых компонентов:

1. CLR (Common Language Runtime)

Среда выполнения программ, которая отвечает за:

- ❑ запуск приложений
- ❑ управление памятью
- ❑ безопасность кода
- ❑ обработку исключений.

2. Библиотеки классов (.NET Class Library)

Большой набор готовых библиотек для работы с:

- ❑ файлами
- ❑ сетью
- ❑ базами данных
- ❑ графикой

Эволюция платформы .NET

Платформа .NET прошла несколько этапов развития.

.NET Framework (2002)

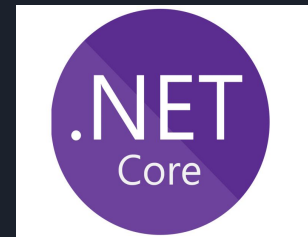
Первая версия платформы, предназначенная для Windows-приложений.



.NET Core (2016)

Новая версия платформы с открытым исходным кодом, обладающая:

- ★ высокой производительностью
- ★ поддержкой Linux и macOS
- ★ возможностью разработки кроссплатформенных приложений.



Позднее эти технологии были объединены в единую платформу **.NET**.



Использование .NET для веб-разработки

Платформа .NET позволяет создавать различные типы приложений, однако одной из наиболее востребованных областей её применения является **разработка серверных веб-приложений и API**.

Для этих задач используется специальный веб-фреймворк — **ASP.NET Core**, который работает поверх платформы .NET.

ASP.NET Core предоставляет инструменты для:

- ❑ создания веб-сайтов
- ❑ разработки серверных приложений
- ❑ построения REST API
- ❑ реализации микросервисной архитектуры

The logo for ASP.NET Core, featuring the text "ASP.NET" in a white serif font, followed by a stylized "Core" where the "C" is a circle with a blue dot in the center, and "re" is in a white sans-serif font.

Таким образом, **.NET выступает в роли базовой платформы**, а **ASP.NET Core используется для разработки backend-части веб-приложений**.



Что такое ASP.NET Core

ASP.NET Core — это технология для создания веб-приложений на платформе **.NET**, развиваемая компанией **Microsoft**.

Для разработки приложений на ASP.NET Core используются языки:

→ **C#**

→ **F#**

ASP.NET Core применяется для создания:

- ❑ веб-приложений
- ❑ серверных сервисов
- ❑ REST API.

История развития ASP.NET

История ASP.NET началась с выхода первой версии **.NET** в **2002** году.

Особенности ранних версий:

- ❑ ASP.NET развивалась параллельно с .NET
- ❑ работала преимущественно на Windows
- ❑ использовала веб-сервер **IIS**

Позднее благодаря проекту **Mono** появилась возможность запускать приложения ASP.NET на Linux.





Появление ASP.NET Core

В **2014 году** компания Microsoft начала развитие новой версии платформы — **ASP.NET Core**.

Основные изменения:

- ❑ платформа стала **кроссплатформенной**
- ❑ проект стал **open source**
- ❑ улучшена производительность
- ❑ фреймворк стал более **легковесным и модульным**

Первый релиз ASP.NET Core вышел в **июне 2016 года**.

Сегодня ASP.NET Core работает на:

- ❑ Windows
- ❑ Linux
- ❑ macOS.

Архитектура ASP.NET Core

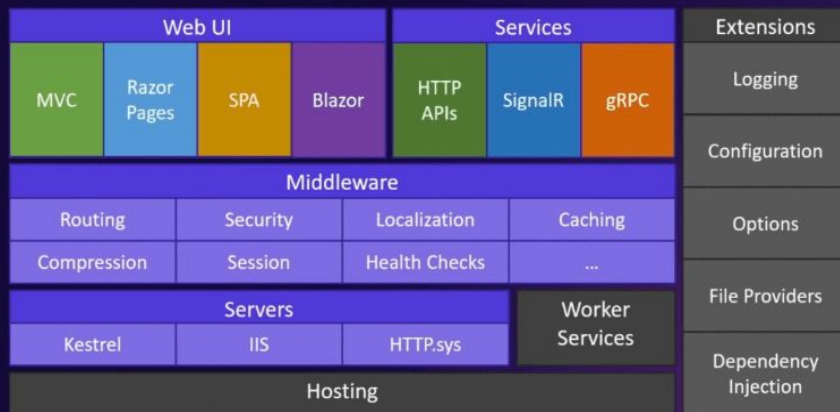
Архитектура ASP.NET Core состоит из нескольких уровней.

Верхний уровень

Технологии взаимодействия с пользователем:

- ❑ MVC
- ❑ Razor Pages
- ❑ SPA (Angular, React, Vue)
- ❑ Blazor
- ❑ HTTP API
- ❑ SignalR
- ❑ gRPC

ASP.NET Core: Unified web framework



Средний уровень

Базовый ASP.NET Core:

- ❑ middleware компоненты
- ❑ обработка HTTP-запросов
- ❑ интеграция с расширениями.

Нижний уровень

Веб-сервер:

- ❑ Kestrel
- ❑ IIS
- ❑ HTTP.sys.

Модели разработки ASP.NET Core

ASP.NET Core поддерживает несколько моделей разработки:

Minimal API

Упрощённая модель разработки API, добавленная в ASP.NET Core 6.

MVC

Архитектура приложения на основе трёх компонентов:

- ❑ Model — работа с данными
- ❑ View — пользовательский интерфейс
- ❑ Controller — обработка запросов.

Razor Pages

Модель разработки, при которой обработка запросов выполняется страницами Razor.

Web API

Реализация REST-подхода для обработки HTTP-запросов:

- ❑ GET
- ❑ POST
- ❑ PUT
- ❑ DELETE.

Blazor

Фреймворк для создания интерактивных веб-приложений с использованием WebAssembly.





Особенности ASP.NET Core

Основные особенности ASP.NET Core:

- ❑ работает поверх платформы **.NET**
- ❑ поддерживает языки **C#** и **F#**
- ❑ является **кроссплатформенным фреймворком**
- ❑ обладает **модульной архитектурой**

Дополнительные возможности:

- загрузка компонентов через **NuGet**
- поддержка различных баз данных
 - ◆ SQL Server
 - ◆ MySQL
 - ◆ PostgreSQL
 - ◆ MongoDB
- расширяемая архитектура компонентов.



Создание первого приложения ASP.NET Core

После рассмотрения архитектуры и возможностей ASP.NET Core важно понять, как создаётся и запускается веб-приложение на практике.

Для разработки приложений ASP.NET Core используются:

- ❑ **.NET CLI**
- ❑ **.NET SDK**
- ❑ текстовый редактор или среда разработки (например **Visual Studio Code**)

Создадим простейшее веб-приложение с помощью **.NET CLI**.

Установка инструментов разработки

Подготовка среды для разработки ASP.NET Core

Шаг 1. Установить .NET SDK

1. Перейти на официальный сайт:
<https://dotnet.microsoft.com/download>
2. Скачать последнюю версию **.NET SDK**
3. Установить её на компьютер.

Проверить установку можно командой в терминале:

```
dotnet --version
```

Если версия отображается — установка выполнена успешно.

Шаг 2. Установить редактор кода

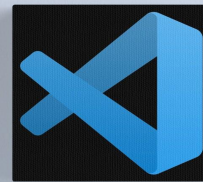
Рекомендуемый редактор:

Visual Studio Code

Скачать можно с сайта:

<https://code.visualstudio.com>

После установки желательно добавить расширение: **C# Dev Kit**





Создание первого приложения ASP.NET Core

После рассмотрения архитектуры и возможностей ASP.NET Core важно понять, как создаётся и запускается веб-приложение на практике.

Для разработки приложений ASP.NET Core используются:

- ❑ **.NET CLI**
- ❑ **.NET SDK**
- ❑ текстовый редактор или среда разработки (например **Visual Studio Code**)

Создадим простейшее веб-приложение с помощью **.NET CLI**.



Создание проекта через .NET CLI

Для создания нового проекта используется команда:

dotnet new web -o helloapp

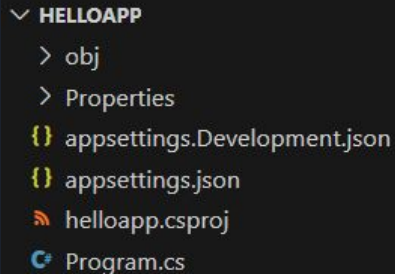
Описание команды:

- **dotnet new** — команда создания нового проекта
- **web** — шаблон ASP.NET Core приложения
- **-o helloapp** — имя проекта и создаваемой папки

После выполнения команды создаётся структура проекта ASP.NET Core.

Для перехода в проект используется команда:

cd helloapp



```
▼ HELLOAPP
  > obj
  > Properties
  {} appsettings.Development.json
  {} appsettings.json
  {} helloapp.csproj
  {} Program.cs
```




Основные файлы проекта

После создания проекта формируется стандартная структура:

Dependencies

подключённые библиотеки и пакеты проекта.

Properties / launchSettings.json

настройки запуска приложения.

appsettings.json

основной файл конфигурации приложения.

appsettings.Development.json

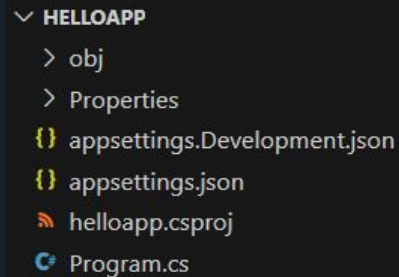
конфигурация для режима разработки.

helloapp.csproj

файл проекта, содержащий настройки сборки.

Program.cs

главный файл приложения, который запускает веб-сервер.



```
▼ HELLOAPP
  > obj
  > Properties
  {} appsettings.Development.json
  {} appsettings.json
  {} helloapp.csproj
  {} Program.cs
```



Файл проекта (.csproj)

Файл **.csproj** содержит основные параметры проекта.

Пример файла проекта:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
  </PropertyGroup>
</Project>
```

Основные параметры:

- **Sdk="Microsoft.NET.Sdk.Web"** — указывает, что проект является веб-приложением
- **TargetFramework** — версия платформы .NET
- настройки сборки и зависимостей



Работа с ASP.NET Core в Visual Studio

Однако для разработки приложений ASP.NET Core также можно использовать среду разработки **Visual Studio 2022**.

Visual Studio упрощает многие задачи разработки:

- Скачать **Visual Studio Community 2022** с официального сайта Microsoft.
- Запустить установщик программы.
- В окне выбора компонентов выбрать рабочую нагрузку:

- создание проектов
- настройку конфигурации
- запуск и отладку приложения
- управление зависимостями.

ASP.NET и разработка веб-приложений

При установке будут автоматически установлены:

- платформа **.NET**
- инструменты разработки ASP.NET Core
- компилятор C#.

Установка Visual Studio

Скачать Инструменты Visual Studio

visualstudio.microsoft.com/ru/downloads/

Visual Studio 2022

Лучшая комплексная среда IDE для разработчиков .NET и C++ в Windows. Полноценный набор инструментов и функций для улучшения и усовершенствования каждого этапа разработки программного обеспечения.

[Заметки о выпуске](#) > [Сравнение выпусков](#) > [Как выполнить установку в автономном режиме](#) >

Community

Эффективная среда IDE предоставляется бесплатно для студентов, участников создания открытого исходного кода и отдельных пользователей

Скачать бесплатно

Professional

Профессиональная интегрированная среда разработки, оптимальная для небольших команд

Бесплатная пробная версия

Enterprise

Масштабируемое, комплексное решение для команд любого размера

Бесплатная пробная версия

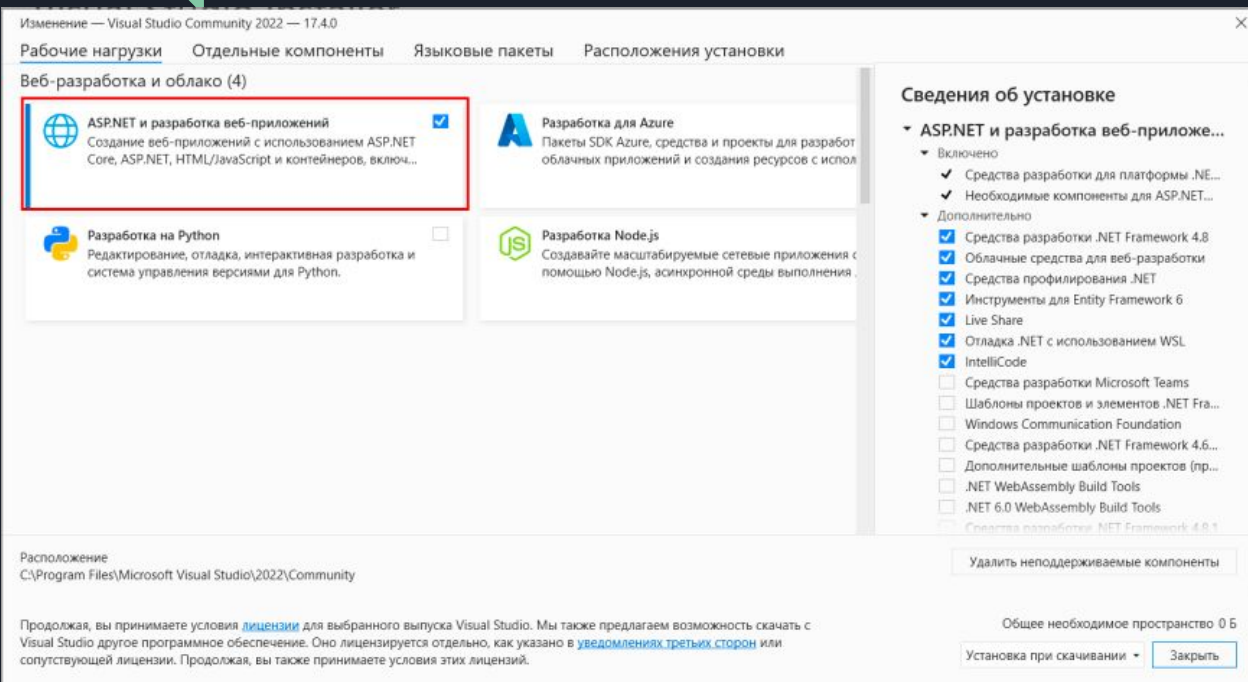
Предварительная версия

Получите ранний доступ к новейшим возможностям, которых еще нет в основной версии

[Подробнее](#) > [Заметки к выпуску](#) >

Обратная связь

Создание проекта в Visual Studio



После запуска **Visual Studio**:

1. На стартовом экране выбрать **Create a new project**
2. В списке шаблонов выбрать:

ASP.NET Core Empty

3. Указать параметры проекта:
 - **Project Name** — имя проекта
 - **Location** — каталог проекта.

После этого среда создаст новый проект ASP.NET Core.

Создание проекта в Visual Studio

Visual Studio 2022

Open recent

As you use Visual Studio, any projects, folders, or files that you open will show up here for quick access.

You can pin anything that you open frequently so that it's always at the top of the list.

Get started



Clone a repository

Get code from an online repository like GitHub or Azure DevOps



Open a project or solution

Open a local Visual Studio project or .sln file



Open a local folder

Navigate and edit code within any folder



Create a new project

Choose a project template with code scaffolding to get started

[Continue without code →](#)

Create a new project

Recent project templates

A list of your recently accessed templates will be displayed here.

Search for templates (Alt+S) 🔍

[Clear](#)

All languages

All platforms

Web

A project template for creating a web app that runs on your desktop and is optionally hosted by an ASP.NET Core app. This template can be used for web apps with rich dynamic user interfaces (UIs).

C#

Linux

macOS

Windows

Cloud

Web



ASP.NET Core Empty

An empty project template for creating an ASP.NET Core application. This template does not have any content in it.

C#

Linux

macOS

Windows

Cloud

Service

Web



ASP.NET Core Web App (Model-View-Controller)

A project template for creating an ASP.NET Core application with example ASP.NET Core MVC Views and Controllers. This template can also be used for RESTful HTTP services.

C#

Linux

macOS

Windows

Cloud

Service

Web



Blazor Server App

A project template for creating a Blazor server app that runs server-side inside an ASP.NET Core app and handles user interactions over a SignalR connection. This template can be used for web apps with rich dynamic user interfaces (UIs).

[Back](#)

[Next](#)

Настройки проекта в Visual Studio

Configure your new project

ASP.NET Core Empty

C#

Linux

macOS

Windows

Cloud

Service

Web

Project name

HelloApp

Additional information

ASP.NET Core Empty

C#

Linux

macOS

Windows

Cloud

Service

Web

Framework ⓘ

.NET 8.0 (Long Term Support)

☒ Configure for HTTPS ⓘ

☐ Enable Docker ⓘ

Docker OS ⓘ

Linux

☐ Do not use top-level statements ⓘ

Back

Create

При создании проекта можно задать дополнительные параметры:

Версия .NET

Выбирается версия платформы .NET, которая будет использоваться проектом.

Configure for HTTPS

Включает использование защищённого протокола HTTPS.

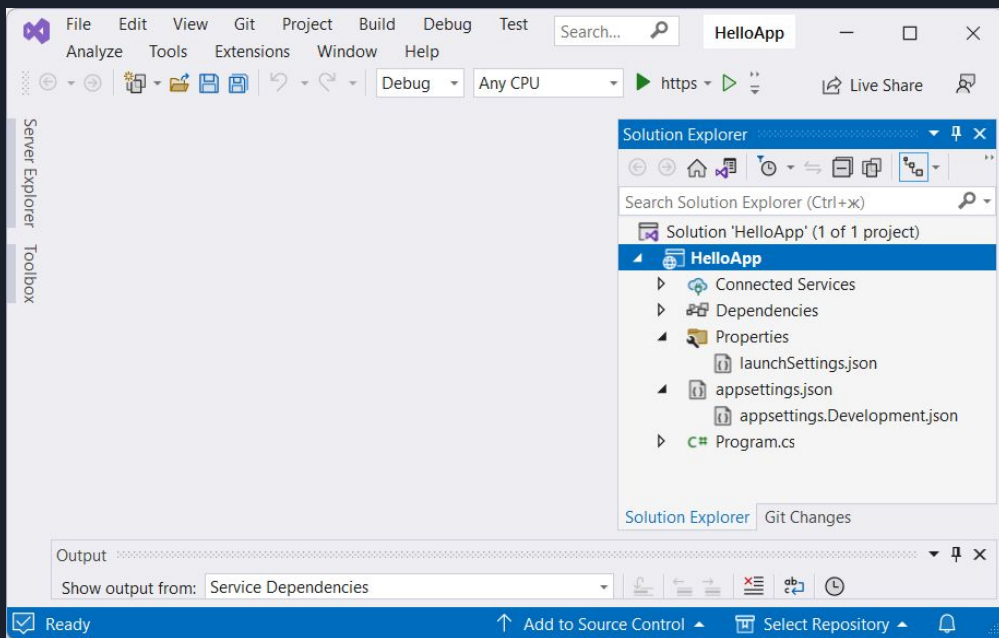
Enable Docker

Позволяет добавить поддержку контейнеризации с помощью Docker.

Do not use top-level statements

Отключает использование выражений верхнего уровня в коде.

Структура проекта и запуск



После создания проекта Visual Studio формирует стандартную структуру:

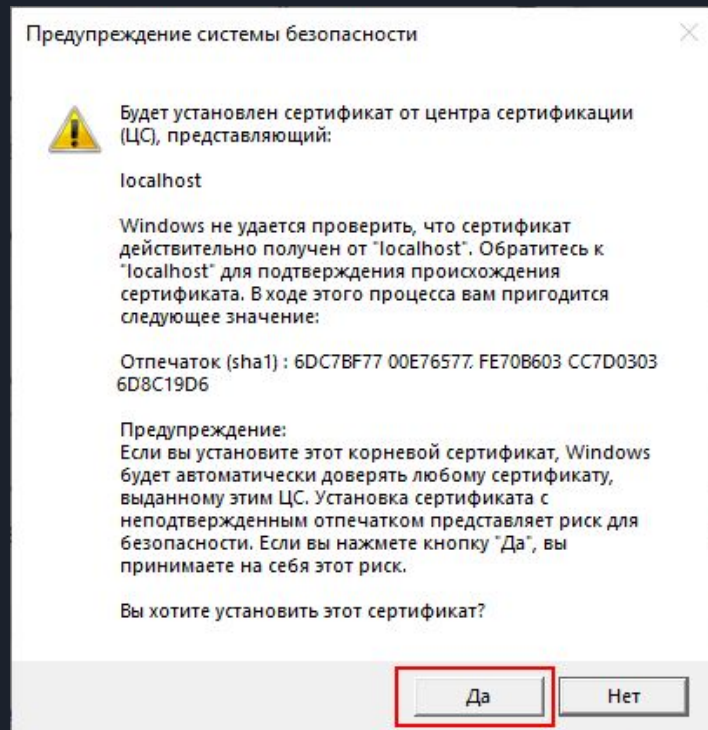
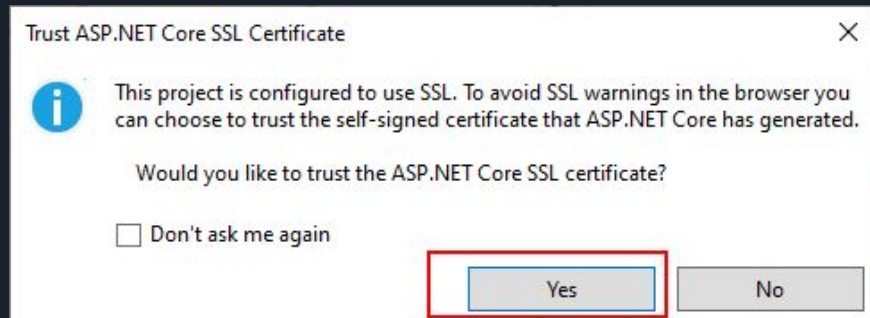
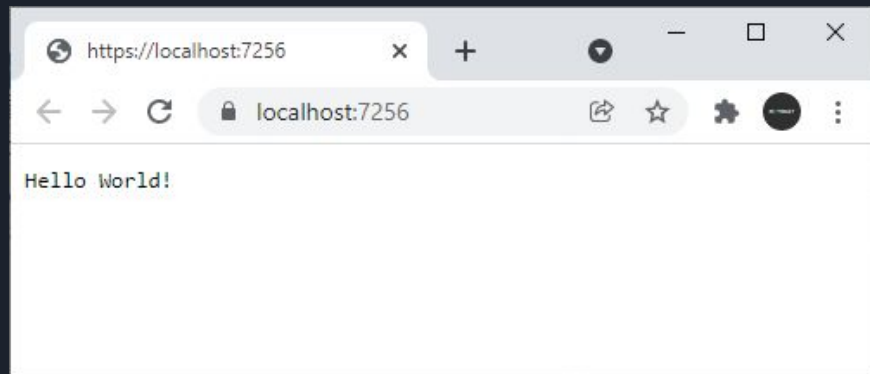
- **Dependencies** — подключённые библиотеки
- **Properties** — настройки запуска
- **appsettings.json** — конфигурация приложения
- **Program.cs** — основной файл запуска приложения.

Для запуска приложения необходимо нажать **Run**.

При первом запуске Visual Studio может предложить установить **SSL-сертификат**.

После запуска откроется браузер, где будет отображаться результат работы программы: **Hello World**

Структура проекта и запуск





Структура проекта ASP.NET Core

После создания проекта ASP.NET Core формируется стандартная структура файлов и каталогов.

Проект включает:

- конфигурационные файлы
- настройки запуска приложения
- зависимости проекта
- основной файл запуска приложения.

Эти элементы обеспечивают работу и конфигурацию веб-приложения.



Основной класс ASP.NET Core приложения Класс WebApplication

В центре приложения **ASP.NET Core** находится класс **WebApplication**.

Простейший пример приложения:

```
var builder = WebApplication.CreateBuilder(args);  
var app = builder.Build();  
  
app.MapGet("/", () => "Hello World!");  
  
app.Run();
```

В этом коде:

- **builder** — объект `WebApplicationBuilder`
(<https://github.com/dotnet/aspnetcore/blob/main/src/DefaultBuilder/src/WebApplicationBuilder.cs>)
- **app** — объект `WebApplication`

Именно объект **WebApplication** представляет запущенное веб-приложение.



Основной класс ASP.NET Core приложения

Класс `WebApplicationBuilder`

Создание приложения начинается с класса **`WebApplicationBuilder`**.

Объект создаётся методом:

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
```

Основные задачи класса:

- настройка конфигурации приложения
- регистрация сервисов
- настройка логирования
- установка окружения приложения
- конфигурация хоста приложения.

Этот класс подготавливает всё необходимое для запуска веб-приложения.



Основные свойства WebApplicationBuilder

Класс **WebApplicationBuilder** содержит ряд важных свойств:

Configuration

управление конфигурацией приложения.

Environment

информация об окружении (Development, Production).

Services

коллекция сервисов приложения.

Logging

настройка логирования.

Host

конфигурация хоста приложения.

WebHost

настройка веб-сервера.

Класс WebApplication и запуск приложения

После настройки `WebApplicationBuilder` создаётся объект приложения:

Класс **WebApplication** отвечает за:

- обработку HTTP-запросов
- настройку маршрутов
- работу с сервисами приложения
- управление жизненным циклом приложения.

Класс `WebApplication` применяет три интерфейса:

- **IHost**: применяется для запуска и остановки хоста, который прослушивает входящие запросы
- **IApplicationBuilder**: применяется для установки компонентов, которые участвуют в обработке запроса
- **IEndpointRouteBuilder**: применяется для установки маршрутов, которые сопоставляются с запросами

Для получения доступа к функциональности приложения можно использовать свойства класса `WebApplication`:

- **Configuration**: представляет конфигурацию приложения в виде объекта **IConfiguration**
- **Environment**: представляет окружение приложения в виде **IWebHostEnvironment**
- **Lifetime**: позволяет получать уведомления о событиях жизненного цикла приложения
- **Logger**: представляет логгер приложения по умолчанию
- **Services**: представляет сервисы приложения
- **Urls**: представляет набор адресов, которые использует сервер

Для запуска приложения используется метод:

```
app.Run();
```

Другие методы управления приложением:

- ❑ **RunAsync()** — асинхронный запуск
- ❑ **Start() / StartAsync()** — запуск приложения
- ❑ **StopAsync()** — остановка приложения.

Пример остановки приложения:

```
await app.StartAsync();  
await Task.Delay(10000);  
await app.StopAsync();
```

В данном примере приложение завершает работу через 10 секунд.

Обработка запросов в ASP.NET Core

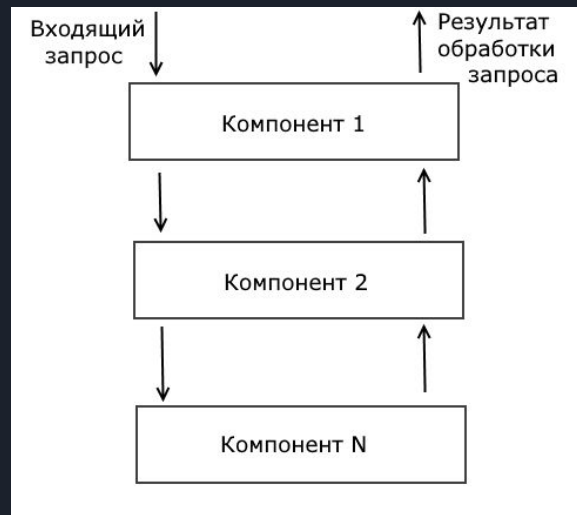
Одна из основных задач веб-приложения — обработка входящих HTTP-запросов.

В **ASP.NET Core** обработка запросов реализована по принципу **конвейера (pipeline)**.

Конвейер состоит из специальных компонентов — **middleware**.

Каждый компонент middleware:

- получает HTTP-запрос
- выполняет обработку
- может передать запрос следующему компоненту
- либо завершить обработку запроса.



Принцип работы middleware

При получении запроса сервер передаёт его первому компоненту middleware.

Далее возможны два варианта:

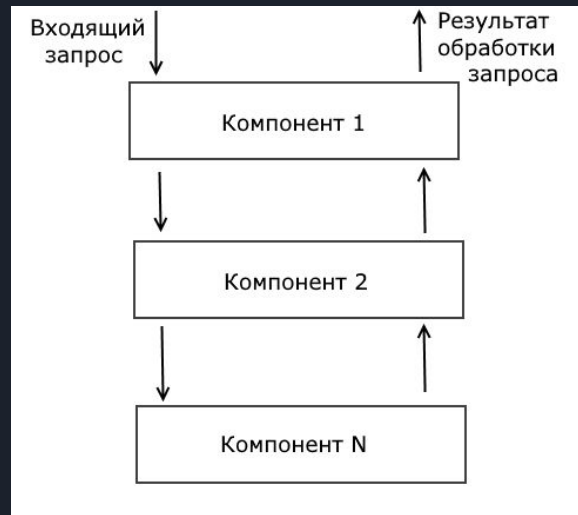
1. Терминальный компонент

middleware завершает обработку запроса и формирует ответ клиенту.

2. Передача дальше

middleware передаёт запрос следующему компоненту конвейера.

После обработки последним компонентом ответ возвращается обратно по цепочке middleware.





HttpContext - Контекст запроса

```
public delegate Task RequestDelegate(HttpContext context);
```

При получении запроса сервер создаёт объект **HttpContext**.

Он содержит всю информацию о текущем запросе и передаётся каждому middleware.

Основные свойства **HttpContext**:

- **Connection**: представляет информацию о подключении, которое установлено для данного запроса
- **Features**: получает коллекцию HTTP-функциональностей, которые доступны для этого запроса
- **Items**: получает или устанавливает коллекцию пар ключ-значение для хранения некоторых данных для текущего запроса
- **Request**: возвращает объект **HttpRequest**, который хранит информацию о текущем запросе
- **RequestAborted**: уведомляет приложение, когда подключение прерывается, и соответственно обработка запроса должна быть отменена
- **RequestServices**: получает или устанавливает объект **IServiceProvider**, который предоставляет доступ к контейнеру сервисов запроса
- **Response**: возвращает объект **HttpResponse**, который позволяет управлять ответом клиенту
- **Session**: хранит данные сессии для текущего запроса
- **TraceIdentifier**: представляет уникальный идентификатор запроса для логов трассировки
- **User**: представляет пользователя, ассоциированного с этим запросом
- **WebSockets**: возвращает объект для управления подключениями WebSocket для данного запроса

Встроенные middleware + подключение middleware

ASP.NET Core предоставляет множество встроенных middleware:

- ❖ **Authentication** — аутентификация пользователей
- ❖ **Authorization** — авторизация
- ❖ **CORS** — поддержка междоменных запросов
- ❖ **HTTPS Redirection** — перенаправление HTTP → HTTPS
- ❖ **Static Files** — обработка статических файлов
- ❖ **Session** — поддержка сессий
- ❖ **Response Caching** — кэширование ответов
- ❖ **Endpoint Routing** — маршрутизация запросов.

Эти компоненты можно подключать к конвейеру приложения.

Компоненты middleware добавляются в конвейер обработки запроса.

Пример подключения middleware:

```
var builder =  
WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.UseWelcomePage();  
  
app.Run();
```

Метод **UseWelcomePage()** подключает middleware

WelcomePageMiddleware, которое возвращает стандартную страницу приветствия.

